অধ্যায় ২০

২০

JavaScript Security Fundamentals, Web Application Security এবং Secure Coding Practices

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একটি Application-এর Threat Model তৈরি করতে: কী রক্ষা করছেন, কার কাছ থেকে, এবং Trust Boundary কোথায়।
- XSS কীভাবে ঘটে তা ব্যাখ্যা করতে, এবং প্রসঙ্গ অনুযায়ী Output Encoding ও নিরাপদ DOM API দিয়ে তা প্রতিরোধ করতে।
- Input Validation ও Output Encoding-এর পার্থক্য বুঝতে, এবং কেন Client-side Validation নিরাপত্তা দেয় না তা ব্যাখ্যা করতে।
- Parameterized Query দিয়ে SQL Injection প্রতিরোধ করতে।
- Authentication ও Authorization আলাদা করতে, এবং প্রতিটি Object-এর জন্য Authorization যাচাই করতে।
- Password নিরাপদে সংরক্ষণ করতে (scrypt, Salt, ধ্রুব-সময়ের তুলনা)।
- Session Cookie-র Flag, CSRF, CORS ও Content Security Policy-র ভূমিকা বুঝতে।
- Secret, Dependency ও Log-এর ঝুঁকি সামলাতে, এবং Prototype Pollution চিনতে।
- একটি স্থানীয় **SecureTrip** API ও Portal তৈরি করে প্রতিটি নিয়ন্ত্রণ Test দিয়ে যাচাই করতে।

পূর্বশর্ত

- অধ্যায় ৮ ও ১১: DOM-এ লেখা দেখানো, Form Validation।
- অধ্যায় ১৫: `fetch`, HTTP Status, JSON।
- অধ্যায় ১৭: Regular Expression দিয়ে Validation, Allowlist।
- অধ্যায় ১৯: Map, Cache।
- Node.js 22.5 বা নতুন (`node:sqlite` Module-এর জন্য)।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার Website-এর কথা ভাবুন। ভুল একটি লাইনের কারণে একজন গ্রাহক আরেকজনের Booking, ফোন নম্বর ও পাসপোর্টের তথ্য দেখে ফেলতে পারেন; একটি Form-এ লুকানো দামের Field বদলে কেউ ১২,৫০০ টাকার Package ১ টাকায় বুক করতে পারেন; একটি Review-এর ভেতরে লুকানো Script প্রতিটি দর্শকের Browser-এ চলতে পারে। এর কোনোটির জন্য উন্নত Hacking দক্ষতা লাগে না—লাগে শুধু Developer-এর একটি ভুল ধারণা।

নিরাপত্তা তাই একটি শেষ ধাপের "Feature" নয়; প্রতিটি Function লেখার সময়ের একটি অভ্যাস। এই অধ্যায়ের মূল প্রশ্ন সব সময় একটি: **এই তথ্য কোথা থেকে এসেছে, এবং আমি কি একে বিশ্বাস করতে পারি?**

> **সতর্কতা:** এই অধ্যায়ের প্রতিটি দুর্বলতার প্রদর্শন শুধু আপনার নিজের কম্পিউটারে, নিরীহ ইনপুট দিয়ে (যেমন একটি Flag বসানো), চালানোর জন্য। অনুমতি ছাড়া অন্যের System-এ এ ধরনের কিছু চেষ্টা করা অনৈতিক এবং অনেক দেশে অপরাধ। Project-এর Server শুধু `127.0.0.1`-এ চলে এবং Deploy করার জন্য নয়।

এই অধ্যায়ে প্রতিটি বিষয় তিনটি স্তরে আলাদা করে দেখানো হয়েছে:

চিহ্ন	অর্থ
শিক্ষামূলক প্রদর্শন	দুর্বলতা কীভাবে কাজ করে, স্থানীয় ও নিরীহ ইনপুটে
প্রস্তাবিত প্রতিরক্ষা	Code-এ যা করা উচিত; SecureTrip-এ প্রয়োগ ও Test করা
Production-এর প্রয়োজন	বাস্তব System-এ যা লাগবে, কিন্তু এই শিক্ষামূলক Project-এ নেই

২০.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`textContent` বনাম `innerHTML` (অধ্যায় ৮)	XSS কীভাবে চলে, প্রসঙ্গভিত্তিক Encoding, CSP
Form Validation (অধ্যায় ১১)	Server-এ পুনরায় Validation, Allowlist, Trust Boundary

আগে শেখা	এই অধ্যায়ে নতুন
Regex Validation ও ReDoS (অধ্যায় ১৭)	Validation বনাম Encoding-এর আলাদা দায়িত্ব
`fetch` ও HTTP Status (অধ্যায় ১৫)	Cookie-র Flag, CSRF, CORS, Security Header
`localStorage` (অধ্যায় ১৫)	কেন Session Token সেখানে রাখা ঝুঁকিপূর্ণ
—	SQL Injection, Password Hashing, Authorization, Prototype Pollution

২০.২ Threat Model ও Trust Boundary

Threat Model চারটি প্রশ্নের উত্তর: আমরা কী তৈরি করছি? কী ভুল হতে পারে? তা নিয়ে কী করব? আমরা কি যথেষ্ট ভালো করেছি? SecureTrip-এর জন্য:

সম্পদ	কে ক্ষতি করতে পারে	কী ঝুঁকি
গ্রাহকের Trip ও ব্যক্তিগত তথ্য	অন্য গ্রাহক	অন্যের তথ্য দেখা (Authorization-এর ত্রুটি)
দাম	যেকোনো গ্রাহক	Request বদলে কম দাম পাঠানো
Account	বাইরের কেউ	Password অনুমান, Session চুরি
অন্য দর্শকের Browser	ক্ষতিকর ইনপুট দেওয়া কেউ	XSS
Database	যেকোনো ইনপুট	SQL Injection

Trust Boundary হলো যেখানে তথ্য কম বিশ্বস্ত এলাকা থেকে বেশি বিশ্বস্ত এলাকায় ঢোকে। সবচেয়ে গুরুত্বপূর্ণটি: **Browser থেকে Server।** ব্যবহারকারীর Browser পুরোপুরি তাঁর নিয়ন্ত্রণে—তিনি DevTools-এ JavaScript বদলাতে, Form-এর `maxlength` মুছে ফেলতে, অথবা Browser ছাড়াই `curl` দিয়ে যেকোনো Request পাঠাতে পারেন। তাই:

> **মনে রাখুন:** Client-side Validation ব্যবহারকারীর সুবিধার জন্য (তাৎক্ষণিক ভুল দেখানো), নিরাপত্তার জন্য নয়। Server-এ পৌঁছানো প্রতিটি তথ্য পুনরায় যাচাই করতে হবে, যেন Browser-এ কোনো যাচাই নেই।

২০.৩ XSS: অন্যের Browser-এ আপনার Page দিয়ে Script

Cross-Site Scripting (XSS) ঘটে যখন ব্যবহারকারীর দেওয়া লেখা Page-এ HTML বা Script হিসেবে ঢুকে পড়ে। তখন সেই Script আপনার Website-এর নামে, প্রতিটি দর্শকের Browser-এ চলে: তাঁর তথ্য পড়তে পারে, তাঁর নামে Request পাঠাতে পারে। নিচের **শিক্ষামূলক প্রদর্শনে** ক্ষতিকর অংশের বদলে নিরীহ একটি Flag বসানো হয়েছে:

xss-demo.js · JavaScript চলবে: Browser

```
const note = '<img src="data:," onerror="window.imgRan = true">';
const scriptNote = '<script>window.scriptRan = true</' + 'script>';

const unsafe = document.createElement('div');
const safe = document.createElement('div');
document.body.append(unsafe, safe);

// শিক্ষামূলক প্রদর্শন: ব্যবহারকারীর লেখা innerHTML-এ (এভাবে করবেন না)।
unsafe.innerHTML = note + scriptNote;
// প্রস্তাবিত প্রতিরক্ষা: লেখা হিসেবে বসানো।
safe.textContent = note;

setTimeout(() => {
  const img = unsafe.querySelector('img');
  console.log(`innerHTML: img Element তৈরি? ${img !== null}`);
  console.log(`innerHTML: onerror চলেছে? ${window.imgRan === true}`);
  console.log(`innerHTML: script চলেছে? ${window.scriptRan === true}`);
  console.log(`textContent: Element তৈরি? ${safe.children.length > 0}`);
}, 300);
```

Output

```
innerHTML: img Element তৈরি? true
innerHTML: onerror চলেছে? true
innerHTML: script চলেছে? false
textContent: Element তৈরি? false
```

দুটি গুরুত্বপূর্ণ পর্যবেক্ষণ। প্রথমত, `innerHTML`-এ ঢোকানো `<script>` চলেনি (HTML-এর নিয়মে এভাবে যোগ করা Script চলে না), কিন্তু একটি নিরীহ চেহারার `<img>`-এর `onerror` ঠিকই চলেছে—তাই "শুধু `<script>` শব্দটি আটকালেই হবে" ধারণাটি ভুল। দ্বিতীয়ত, `textContent` কোনো Element তৈরি করে না; লেখাটি হুবহু লেখা হিসেবেই দেখায়।

XSS-এর তিন ধরন: **Stored** (ক্ষতিকর লেখা Database-এ সংরক্ষিত হয়ে সবাইকে দেখানো হয়, যেমন Review), **Reflected** (URL-এর অংশ সরাসরি Page-এ ফেরত দেখানো), এবং **DOM-based** (Client-এর JavaScript নিজেই `location.hash` বা অন্য উৎস থেকে নিয়ে `innerHTML`-এ বসায়)। প্রতিরক্ষা তিনটিতেই এক: অবিশ্বস্ত তথ্য কখনো Code বা Markup হিসেবে নয়।

প্রস্তাবিত প্রতিরক্ষা:

- লেখার জন্য `textContent`, Attribute-এর জন্য `setAttribute` বা Property; `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write` অবিশ্বস্ত তথ্যে নয়।
- সত্যিই ব্যবহারকারীর HTML দেখাতে হলে (যেমন Rich Text) একটি পরীক্ষিত Sanitizer Library (যেমন DOMPurify)—নিজের লেখা Regex নয়।

- React, Vue-র মতো Framework স্বয়ংক্রিয়ভাবে Escape করে; কিন্তু `dangerouslySetInnerHTML` বা `v-html` সেই সুরক্ষা বন্ধ করে।
- দ্বিতীয় স্তর হিসেবে Content Security Policy (অনুচ্ছেদ ২০.৯)।

২০.৪ Input Validation বনাম Output Encoding

দুটি আলাদা কাজ, এবং একটি অন্যটির বিকল্প নয়।

- **Input Validation** প্রশ্ন করে: "এই তথ্য কি আমাদের নিয়মে বৈধ?" (যাত্রী ১ থেকে ৮, তারিখ ক্যালেন্ডারে আছে)। Server-এ, **Allowlist** দিয়ে: কী গ্রহণযোগ্য তা বলুন, কী নিষিদ্ধ তার তালিকা নয়।
- **Output Encoding** প্রশ্ন করে: "এই তথ্য যেখানে বসাচ্ছি, সেই প্রসঙ্গে কি এটি শুধু তথ্য হিসেবেই থাকবে?" এটি ঘটে **ব্যবহারের মুহূর্তে**, এবং প্রসঙ্গভেদে আলাদা।

একটি নোট সম্পূর্ণ বৈধ হতে পারে ("আমার বাজেট <১৫০০০ টাকা & দুটি রুম"), তবু HTML-এ Encoding ছাড়া বসালে ভেঙে যাবে। তাই Validation পাস করা তথ্যও Output-এর সময় Encode করতে হয়।

encoding.js · JavaScript চলবে: Browser Console ও Node.js

```
const HTML_ESCAPES = {
  '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;',
};
const escapeHtml = (text) =>
  String(text).replace(/[&<>"']/g, (ch) => HTML_ESCAPES[ch]);

function isSafeLink(url) {
  try {
    const { protocol } = new URL(url, 'https://trip.example');
    return ['https:', 'http:'].includes(protocol);
  } catch {
    return false;
  }
}

const name = 'Rafi" onfocus="window.flag=1';
console.log(`<input value="${escapeHtml(name)}">`);
console.log(`<p>${escapeHtml('বাজেট <15000 & দুটি রুম')}</p>`);
console.log(`/search?q=${encodeURIComponent('Cox & Sylhet = সাগর')}`);
console.log(isSafeLink('javascript:alert(1)'), isSafeLink('/trips/7'));
```

Output

```
<input value="Rafi&quot; onfocus=&quot;window.flag=1">
<p>বাজেট &lt;15000 &amp; দুটি রুম</p>
/search?q=Cox%20%26%20Sylhet%20%3D%20%E0%A6%B8%E0%A6%BE%E0%A6%97%E0%A6%B0
false true
```

প্রসঙ্গ	সঠিক উপায়
HTML-এর লেখা	`textContent`; Server-এ Template-এ HTML Escape
HTML Attribute	`setAttribute`; Template-এ উদ্ধৃতিচিহ্নসহ Attribute ও Escape
URL-এর অংশ	`encodeURIComponent`, অথবা `URLSearchParams`
Link-এর `href`	Protocol যাচাই (`javascript:` URL একটি XSS-এর পথ)
SQL	Parameterized Query (অনুচ্ছেদ ২০.৫)
Shell Command	Shell নয়; Argument-এর Array (`execFile`)

`escapeHtml` উদ্ধৃতিচিহ্নসহ Attribute-এর জন্য নিরাপদ, কিন্তু `<script>` Block-এর ভেতরে, CSS-এ বা উদ্ধৃতিচিহ্নহীন Attribute-এ নয়—প্রতিটি প্রসঙ্গের নিজস্ব নিয়ম। এ কারণেই হাতে Encoding-এর চেয়ে নিরাপদ DOM API ও স্বয়ংক্রিয় Escape করা Template System ভালো।

২০.৫ SQL Injection ও Parameterized Query

ব্যবহারকারীর লেখা দিয়ে SQL জোড়া লাগালে লেখাটিই SQL-এর অংশ হয়ে যায়। নিচের **শিক্ষামূলক প্রদর্শন** একটি In-memory Database-এ চলে, যা Program শেষে মুছে যায় (Node.js-এর নিজস্ব `node:sqlite`; চালানোর সময় এটি "Experimental" বলে একটি সতর্কবার্তা দেখায়):

sql-injection.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE trips (id INTEGER, owner TEXT, place TEXT);
  INSERT INTO trips VALUES (1, 'rafi', 'Sylhet'), (2, 'mina', 'Cox'),
    (3, 'sara', 'Bogura');`);

const owner = "rafi' OR '1'='1";

// শিক্ষামূলক প্রদর্শন: লেখা জোড়া লাগানো (এভাবে করবেন না)।
const query = `SELECT id FROM trips WHERE owner = '${owner}'`;
const unsafe = db.prepare(query).all();
// প্রস্তাবিত প্রতিরক্ষা: Parameter; মান কখনো SQL-এর লেখা হয় না।
const safe = db.prepare('SELECT id FROM trips WHERE owner = ?').all(owner);

console.log(query);
console.log('জোড়া লাগানো:', unsafe.map((row) => row.id).join(', '));
console.log('Parameterized:', safe.length, 'টি সারি');
```

Output

```
SELECT id FROM trips WHERE owner = 'rafi' OR '1'='1'
জোড়া লাগানো: 1, 2, 3
Parameterized: 0 টি সারি
```

ইনপুটের `'` চিহ্ন SQL-এর লেখা বন্ধ করে দিয়েছে, এবং `OR '1'='1'` শর্তটি সব সারির জন্য সত্য—শুধু rafi-র নয়, সবার Trip বেরিয়ে এসেছে। Parameter (`?`) ব্যবহারে Database মানটিকে শুধু মান হিসেবে পায়: "rafi' OR '1'='1" নামে কোনো মালিক নেই, তাই শূন্য সারি।

Parameter দিয়ে মান পাঠানো যায়, কিন্তু Table বা Column-এর নাম নয়। ব্যবহারকারী Sort-এর Column বেছে নিলে একটি Allowlist থেকে মেলান (`{ date: 'start_date', price: 'total' }`), কখনো সরাসরি বসাবেন না। ORM বা Query Builder সাধারণত Parameter ব্যবহার করে, কিন্তু তাদের "raw" Query-তেও একই নিয়ম। একই নীতি NoSQL-এ: MongoDB-তে `{ email: req.body.email }`-এ Body থেকে `{ "$ne": null }` এলে তা একটি Operator হয়ে যায়; তাই ধরন যাচাই (`typeof email === 'string'`)।

২০.৬ Authentication বনাম Authorization

- **Authentication (AuthN):** আপনি কে? (Login, Session)
- **Authorization (AuthZ):** আপনি কি **এই নির্দিষ্ট কাজটি, এই নির্দিষ্ট Object-এ** করতে পারেন?

সবচেয়ে সাধারণ ত্রুটি দ্বিতীয়টিতে: Login যাচাই করা হলো, কিন্তু Trip-টি এই ব্যবহারকারীর কি না তা নয়। তখন `/api/trips/T2`-এর জায়গায় `/api/trips/T3` লিখলেই অন্যের তথ্য—এর নাম **IDOR** (Insecure Direct Object Reference), এবং OWASP-এর তালিকায় এক নম্বর ঝুঁকি "Broken Access Control"-এর অংশ।

authz.js · JavaScript — চলবে: Browser Console ও Node.js

```
const trips = [
  { id: 'T1', ownerId: 1, place: 'Sylhet' },
  { id: 'T2', ownerId: 2, place: 'Cox' },
];

function readTrip(user, id) {
  if (!user) {
    return '401 Login প্রয়োজন';
  }
  const trip = trips.find((item) => item.id === id);
  if (!trip || (trip.ownerId !== user.id && user.role !== 'admin')) {
    return '404 পাওয়া যায়নি';
  }
  return `200 ${trip.place}`;
}

const rafi = { id: 1, role: 'customer' };
const admin = { id: 9, role: 'admin' };
```

```
console.log(readTrip(null, 'T1'));
console.log(['T1', 'T2', 'T9'].map((id) => readTrip(rafi, id)).join(' | '));
console.log(readTrip(admin, 'T2'));
```

Output

```
401 Login প্রয়োজন
200 Sylhet | 404 পাওয়া যায়নি | 404 পাওয়া যায়নি
200 Cox
```

অন্যের Trip (`T2`) এবং অস্তিত্বহীন Trip (`T9`)—দুটিতেই একই উত্তর। 403 দিলে আক্রমণকারী জানতেন T2 আছে; 404 অস্তিত্বও লুকায়। আরেকটি নিয়ম: **মালিকের পরিচয় আসে Session থেকে, Request Body থেকে কখনো নয়।** Body-তে `ownerId: 2` পাঠালেই কেউ অন্যের নামে Trip তৈরি করতে পারলে সেটি একই ত্রুটি। UI-তে Button লুকানো Authorization নয়; প্রতিটি Server Endpoint-এ যাচাই।

২০.৭ Password: Hash, Salt এবং ধ্রুব-সময়ের তুলনা

Password কখনো সরাসরি সংরক্ষণ নয়, এবং SHA-256-এর মতো দ্রুত Hash-ও নয়: দ্রুত Hash একটি সাধারণ GPU-তে প্রতি সেকেন্ডে শত কোটি অনুমান যাচাই করতে দেয়। দরকার **ইচ্ছাকৃতভাবে ধীর ও Memory-নির্ভর** Password Hash: **scrypt** (Node.js-এ অন্তর্নির্মিত), Argon2id বা bcrypt। প্রতিটি Password-এর আলাদা এলোমেলো **Salt** নিশ্চিত করে যে দুজনের একই Password থাকলেও Hash আলাদা, এবং আগে থেকে হিসাব করা তালিকা (Rainbow Table) কাজ করে না।

password-demo.mjs · JavaScript চলবে: Node.js

```
import { hashPassword, verifyPassword }
  from '../project/secure-trip/src/passwords.mjs';

const stored = await hashPassword('Bogura-2026');
const [scheme, salt, key] = stored.split('$');
console.log(scheme, `salt ${salt.length / 2} byte`, `key ${key.length / 2} byte`);

console.log(await verifyPassword('Bogura-2026', stored));
console.log(await verifyPassword('bogura-2026', stored));
console.log(stored === await hashPassword('Bogura-2026'));
```

Output

```
scrypt salt 16 byte key 32 byte
true
false
false
```

শেষ লাইনটি লক্ষ করুন: একই Password-এর দ্বিতীয় Hash ভিন্ন, কারণ Salt নতুন। যাচাইয়ের সময় সংরক্ষিত Salt দিয়ে আবার হিসাব করে তুলনা হয়, এবং তুলনা `timingSafeEqual` দিয়ে: সাধারণ `===` প্রথম অমিল Byte-এ থেমে যায়, তাই উত্তরের সময় থেকে তথ্য ফাঁস হতে পারে। Project-এর Code অনুচ্ছেদ ২০.১২-এ।

Production-এর প্রয়োজন: scrypt বা Argon2id-এর Parameter (Memory, পুনরাবৃত্তি) সময়ের সঙ্গে বাড়ানো ও পুরোনো Hash নতুন Login-এ হালনাগাদ; ফাঁস হওয়া Password-এর তালিকার সঙ্গে মিলিয়ে দেখা; Multi-factor Authentication; নিরাপদ Password Reset (এককালীন, স্বল্পমেয়াদি Token)।

২০.৮ Session, Cookie এবং CSRF

Login-এর পর Server একটি দীর্ঘ এলোমেলো **Session ID** তৈরি করে Cookie-তে পাঠায়; Browser পরের প্রতিটি Request-এ তা ফেরত পাঠায়। Cookie-র Flag-গুলোই তার নিরাপত্তা:

Flag	কাজ
`HttpOnly`	JavaScript (`document.cookie`) পড়তে পারে না; XSS হলেও Session ID চুরি কঠিন
`Secure`	শুধু HTTPS-এ পাঠানো হয়
`SameSite=Strict` বা `Lax`	অন্য Site থেকে শুরু হওয়া Request-এ Cookie পাঠানো হয় না (Lax-এ সাধারণ Link-এ পাঠায়)
`Path=/` , মেয়াদ	কোথায় ও কতদিন বৈধ

Session Token `localStorage`-এ রাখলে যেকোনো XSS তা পড়ে নিতে পারে; `HttpOnly` Cookie-তে পারে না। (XSS থাকলে আক্রমণকারী তবু Page থেকেই Request পাঠাতে পারেন— `HttpOnly` ক্ষতি কমায়, XSS-এর সমাধান নয়।)

CSRF (Cross-Site Request Forgery): Browser Cookie স্বয়ংক্রিয়ভাবে পাঠায়, এমনকি অন্য একটি Website থেকে শুরু হওয়া Request-এও। তাই একটি ক্ষতিকর Page লুকিয়ে আপনার Site-এ একটি Form Submit করালে, Login থাকা ব্যবহারকারীর Cookie-সহ তা পৌঁছায়। প্রতিরক্ষার স্তর:

১. `SameSite` **Cookie**—আধুনিক Browser-এ সবচেয়ে বড় সুরক্ষা।
২. **CSRF Token:** Session-এর সঙ্গে যুক্ত একটি গোপন মান, যা Page-এর JavaScript একটি Header-এ পাঠায়। অন্য Site সেই মান পড়তে পারে না (Same-Origin Policy), তাই পাঠাতেও পারে না।
৩. `Origin` **Header যাচাই:** অবস্থা বদলানো Request কোন Site থেকে এসেছে।
৪. অবস্থা বদলানো কাজ কখনো `GET`-এ নয়।

SecureTrip তিনটিই করে (অনুচ্ছেদ ২০.১২)। এখানে একটি সতর্কতা: এই বইয়ের Test-গুলো Token ও Origin যাচাই প্রমাণ করে, কিন্তু Browser আসলে অন্য Site থেকে `SameSite` Cookie পাঠায় কি না, তা দুটি আলাদা Domain ছাড়া পরীক্ষা করা যায় না—এর জন্য আমরা Browser-এর নথিভুক্ত আচরণের ওপর নির্ভর করছি।

২০.৯ CORS, Same-Origin Policy এবং Content Security Policy

Same-Origin Policy: একটি Page-এর JavaScript অন্য Origin-এর (Protocol + Host + Port) Response পড়তে পারে না। **CORS** এই নিয়ম শিথিল করার উপায়: Server `Access-Control-Allow-Origin` Header দিয়ে জানায় কোন Origin তার Response পড়তে পারবে।

> **সতর্কতা:** CORS একটি নিরাপত্তা-নিয়ন্ত্রণ নয় যা আপনার API রক্ষা করে; এটি Browser-কে বলে কাদের **পড়তে দেওয়া** হবে। `curl` বা অন্য Server CORS মানে না। `Access-Control-Allow-Origin: *` একটি Public API-র জন্য ঠিক, কিন্তু Cookie-সহ API-তে কখনো Request-এর `Origin` হুবহু ফেরত পাঠানো নয়। Authentication ও Authorization সব সময় আলাদা।

SecureTrip-এর Page ও API একই Origin-এ, তাই কোনো CORS Header নেই—সবচেয়ে নিরাপদ অবস্থা।

Content Security Policy (CSP) একটি Response Header, যা Browser-কে বলে কোন উৎসের Script, Style ইত্যাদি চলতে পারবে। `script-src 'self'` মানে শুধু একই Origin-এর Script File; Page-এর ভেতরে লেখা Inline Script ও `onerror="..."`-এর মতো Attribute চলবে না। ফলে কোনোভাবে XSS ঢুকে পড়লেও তা চলার সম্ভাবনা অনেক কমে। Chrome-এ যাচাই করুন: Page-এ একটি `<meta>` CSP, একটি Inline Script, এবং একটি বাইরের Script File।

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta http-equiv="Content-Security-Policy" content="script-src 'self'">
  <title>CSP পরীক্ষা</title>
</head>
<body>
  <script>window.inlineRan = true;</script>
  <script src="check.js"></script>
</body>
</html>
```

check.js · JavaScript চলবে: Browser

```
const holder = document.createElement('div');
holder.innerHTML = '<img src="data:," onerror="window.handlerRan = true">';
document.body.append(holder);

setTimeout(() => {
  console.log(`Inline script চলেছে? ${window.inlineRan === true}`);
  console.log(`onerror Attribute চলেছে? ${window.handlerRan === true}`);
  console.log('বাইরের File (check.js) চলেছে? true');
}, 200);
```

Error Message

```
Inline script চলেছে? false
onerror Attribute চলেছে? false
বাইরের File (check.js) চলেছে? true
Executing inline script violates the following Content Security Policy
Executing inline event handler violates the following Content Security Policy
```

শেষ দুটি লাইন Chrome-এর নিজের CSP বার্তার শুরু (পুরো বার্তায় কোন Directive আটকেছে এবং কীভাবে অনুমতি দেওয়া যায়, তাও লেখা থাকে)। অনুচ্ছেদ ২০.৩-এ যে `onerror` চলেছিল, CSP-র অধীনে একই Code চলেনি। বাস্তবে CSP একটি **Header** হিসেবে পাঠানো ভালো (`<meta>` কিছু Directive সমর্থন করে না); SecureTrip তাই করে। অন্যান্য দরকারি Header:

Header	কাজ
`Strict-Transport-Security`	শুধু HTTPS (Production-এ, HTTPS চালু থাকলে)
`X-Content-Type-Options: nosniff`	Browser যেন File-এর ধরন অনুমান না করে
`Referrer-Policy`	অন্য Site-এ যাওয়ার সময় URL কতটা জানানো হবে
CSP-র `frame-ancestors 'none'`	অন্য Site আপনার Page Frame-এ বসাতে পারবে না (Clickjacking)

২০.১০ Prototype Pollution

JavaScript-এ প্রায় প্রতিটি Object `Object.prototype` থেকে Property উত্তরাধিকারে পায় (অধ্যায় ১৩)। কোনো Code যদি ব্যবহারকারীর JSON-এর Key ধরে Object-এ লেখে, এবং Key হয় `__proto__`, তবে লেখাটি সব Object-এর সাধারণ Prototype-এ পড়ে—**Prototype Pollution**। নিচের **শিক্ষামূলক প্রদর্শন** একটি নিজস্ব Process-এ চলে, এবং শেষে দূষণ মুছে ফেলে:

pollution.js · JavaScript চলবে: Node.js

```
function naiveMerge(target, source) {
  for (const key of Object.keys(source)) {
```

```
    const value = source[key];
    if (typeof value === 'object' && value !== null) {
      target[key] ??= {};
      naiveMerge(target[key], value);
    } else {
      target[key] = value;
    }
  }
  return target;
}

const body = JSON.parse('{ "theme": "dark", "__proto__": { "isAdmin": true } }');
console.log(Object.keys(body));

naiveMerge({}, body);
const unrelated = {};
console.log('সম্পর্কহীন Object-এ isAdmin:', unrelated.isAdmin);
delete Object.prototype.isAdmin;

const BLOCKED = new Set(['__proto__', 'constructor', 'prototype']);
function safeMerge(target, source) {
  for (const [key, value] of Object.entries(source)) {
    if (BLOCKED.has(key)) {
      continue;
    }
    target[key] = typeof value === 'object' && value !== null
      ? safeMerge(Object.hasOwn(target, key) ? target[key] : {}, value)
      : value;
  }
  return target;
}

console.log(safeMerge({}, body), ({}).isAdmin);
```

Output

```
[ 'theme', '__proto__' ]
সম্পর্কহীন Object-এ isAdmin: true
{ theme: 'dark' } undefined
```

`JSON.parse` নিজে নিরাপদ: সে `__proto__` নামে একটি সাধারণ Key তৈরি করে। বিপদ আসে `target['__proto__']` পড়ায়, যা আসলে `Object.prototype`, এবং তার ভেতরে লেখায়। এরপর Program-এর **যেকোনো** Object—এমনকি যেটির সঙ্গে ব্যবহারকারীর কোনো সম্পর্ক নেই—`isAdmin: true` দেখায়; `if (user.isAdmin)`-এর মতো Code বিপজ্জনক হয়ে ওঠে।

প্রতিরক্ষা: বিপজ্জনক Key বাদ; Dictionary হিসেবে `Map` বা `Object.create(null)` (যার Prototype নেই); Property পড়ার সময় `Object.hasOwn`; এবং সবচেয়ে ভালো—SecureTrip-এর মতো, Body থেকে শুধু নির্দিষ্ট Field নেওয়া (Allowlist), পুরো Object মেশানো নয়। পুরোনো Deep-merge Library-গুলোতে এ ধরনের ত্রুটির ইতিহাস আছে, যা পরের অনুচ্ছেদের বিষয়।

২০.১১ Secret, Dependency এবং Log

Secret (API Key, Database Password, Session-এর Signing Key):

- Code-এ বা Git-এ কখনো নয়; Environment Variable বা Secret Manager থেকে। Git-এ একবার Commit হলে পরে মুছলেও History-তে থেকে যায়—তখন Secret বদলাতে (Rotate) হবে।
- Browser-এর JavaScript-এ পাঠানো কোনো কিছুই গোপন নয়: Bundle-এর ভেতরের API Key যে কেউ দেখতে পারে।
- Program শুরুতেই প্রয়োজনীয় Secret আছে কি না যাচাই করে স্পষ্ট বার্তায় থেমে যাওয়া, পরে হঠাৎ ব্যর্থ হওয়ার চেয়ে ভালো।

secrets.js · JavaScript — চলবে: Browser Console ও Node.js

```
function requireSecret(name, env) {
  const value = env[name];
  if (typeof value !== 'string' || value.length < 32) {
    throw new Error(`${name} সেট করা নেই বা ৩২ অক্ষরের কম`);
  }
  return value;
}

try {
  requireSecret('SESSION_SECRET', {});
} catch (error) {
  console.log(error.message);
}
const env = { SESSION_SECRET: 'k'.repeat(48) };
console.log(requireSecret('SESSION_SECRET', env).length);
```

Output

```
SESSION_SECRET সেট করা নেই বা ৩২ অক্ষরের কম
48
```

Dependency: একটি সাধারণ Project শত শত npm Package ব্যবহার করে, এবং প্রতিটির Code আপনার Server-এ আপনার অধিকারে চলে। `npm audit` জানা দুর্বলতা দেখায়; `package-lock.json` Commit করে সংস্করণ স্থির রাখুন; নতুন Package যোগের আগে তার রক্ষণাবেক্ষণ ও জনপ্রিয়তা দেখুন, এবং নামের বানান যাচাই করুন (নকল নামের ক্ষতিকর Package বা Typosquatting আছে)। SecureTrip কোনো বাইরের Package ব্যবহার করে না—শুধু Node.js-এর অন্তর্নির্মিত Module।

Log: নিরাপত্তার ঘটনা (ব্যর্থ Login, প্রত্যাখ্যাত Request) Log করুন, কিন্তু Password, Token, Cookie বা পূর্ণ Card নম্বর কখনো নয়। SecureTrip-এর `redact` Log-এ যাওয়ার আগে সংবেদনশীল Key মুছে দেয়। ব্যবহারকারীকে Error-এর বিস্তারিত (Stack Trace, SQL) দেখাবেন না; তা শুধু Server-এর Log-এ।

২০.১২ Project: SecureTrip

SecureTrip একটি স্থানীয় Trip Request Portal: Node.js-এর `node:http` দিয়ে একটি ছোট API, `node:sqlite`-এর In-memory Database, এবং একটি সাধারণ Browser UI। গ্রাহক Login করে Package, যাত্রীসংখ্যা ও তারিখ দিয়ে Trip-এর অনুরোধ পাঠান এবং নিজের অনুরোধগুলো দেখেন; Admin সব দেখেন। কোনো বাইরের Package নেই।

> **সতর্কতা:** SecureTrip একটি **শিক্ষামূলক Demo**। এটি শুধু `127.0.0.1`-এ চলে, তথ্য Memory-তে থাকে এবং বন্ধ করলে মুছে যায়, এবং এটি Production-এ ব্যবহারের উপযুক্ত নয়। প্রতিটি নিয়ন্ত্রণ Test দিয়ে যাচাই করা হয়েছে, কিন্তু কোনো স্বাধীন Security Review বা Penetration Test হয়নি। নিচের তালিকায় দেখুন বাস্তব System-এ আর কী লাগবে।

নিয়ন্ত্রণ	SecureTrip-এ	Production-এর প্রয়োজন
Password	scrypt, Salt, `timingSafeEqual`	Argon2id বা শক্তিশালী Parameter, MFA, Reset-এর ব্যবস্থা
Session	এলোমেলো ID, HttpOnly, SameSite=Strict, ৩০ মিনিট	`Secure` (HTTPS), স্থায়ী Session Store, Login-এ নতুন ID
CSRF	Token Header ও Origin যাচাই	একই, সব অবস্থা-বদলানো Route-এ
Authorization	প্রতিটি Trip-এ মালিক বা Admin	Role ও অনুমতির পূর্ণ নকশা, Audit Log
দাম	Server-এর Catalog	Database-এর দাম, Transaction, Payment Provider
Validation	Server-এ Allowlist, Body-র আকার ও ধরন	একই, প্রতিটি Endpoint-এ Schema
SQL	Parameterized Query	একই; Database ব্যবহারকারীর ন্যূনতম অধিকার
Login-এর সীমা	Email প্রতি ১৫ মিনিটে ৫ বার (Memory-তে)	IP ও Account-ভিত্তিক, ভাগ করা Store, CAPTCHA
Header	CSP, nosniff, Referrer-Policy, frame-ancestors	HSTS, HTTPS, Reverse Proxy-র নিয়ম
Log	Redaction, সাধারণ Error বার্তা	কেন্দ্রীয় Log, সতর্কতা, সংরক্ষণের নীতি
Dependency	নেই	`npm audit`, Lock File, নিয়মিত হালনাগাদ

File-এর কাঠামো

Folder-এর কাঠামো

```
secure-trip/
├── server.mjs            চালু করে (127.0.0.1:3020)
├── src/
│   ├── start.mjs         সব অংশ জুড়ে Server তৈরি
│   ├── app.mjs           Route, AuthN, AuthZ, CSRF
│   ├── http.mjs          Security Header, JSON পড়া, Cookie
│   ├── passwords.mjs     scrypt Hash ও যাচাই
│   ├── sessions.mjs      Session ও CSRF Token
│   ├── validate.mjs      Trip-এর Allowlist Validation, Catalog
│   ├── db.mjs            SQLite, Parameterized Query
│   ├── limiter.mjs       ব্যর্থ Login-এর সীমা
│   └── log.mjs           Redaction-সহ Log
├── public/               index.html, app.mjs, style.css
└── tests/                units.test.mjs, api.test.mjs
```

src/passwords.mjs ও src/sessions.mjs

passwords.mjs · JavaScript চলবে: Node.js

```
import { scrypt, randomBytes, timingSafeEqual } from 'node:crypto';
import { promisify } from 'node:util';

const scryptAsync = promisify(scrypt);
const KEY_LENGTH = 32;

// সংরক্ষণের রূপ "scrypt$<salt>$<hash>"; প্রতিটি Password-এর নিজস্ব Salt।
export async function hashPassword(password) {
  const salt = randomBytes(16);
  const key = await scryptAsync(password.normalize('NFC'), salt, KEY_LENGTH);
  return `scrypt$${salt.toString('hex')}$${key.toString('hex')}`;
}

export async function verifyPassword(password, stored) {
  const [scheme, saltHex, keyHex] = String(stored).split('$');
  const expected = Buffer.from(keyHex ?? '', 'hex');
  if (scheme !== 'scrypt' || !saltHex || expected.length !== KEY_LENGTH) {
    return false;
  }
  const salt = Buffer.from(saltHex, 'hex');
  const actual = await scryptAsync(password.normalize('NFC'), salt, KEY_LENGTH);
  return timingSafeEqual(actual, expected);
}
```

sessions.mjs · JavaScript চলবে: Node.js

```
import { randomBytes, timingSafeEqual } from 'node:crypto';

const token = () => randomBytes(32).toString('base64url');

// দৈর্ঘ্য আলাদা হলে সঙ্গে সঙ্গে false; সমান হলে ধ্রুব-সময়ের তুলনা।
export function safeEqual(a, b) {
  const left = Buffer.from(String(a));
  const right = Buffer.from(String(b));
  return left.length === right.length && timingSafeEqual(left, right);
}

export function createSessions(options = {}) {
  const { ttlMs = 30 * 60 * 1000, now = Date.now } = options;
  const sessions = new Map();
  return {
    create(user) {
      const id = token();
      const csrf = token();
      const expires = now() + ttlMs;
      const { id: userId, email, role } = user;
      sessions.set(id, { userId, email, role, csrf, expires });
      return { id, csrf };
    },
    get(id) {
      const session = sessions.get(id);
      if (!session) {
        return null;
      }
      if (now() >= session.expires) {
        sessions.delete(id);
        return null;
      }
      return session;
    },
    destroy(id) {
      sessions.delete(id);
    },
  };
}
```

- **`verifyPassword`** সংরক্ষিত মান বিকৃত হলে Exception নয়, `false` দেয়; Key-এর দৈর্ঘ্য না মিললে `timingSafeEqual` চালানোর আগেই ফেরে (এই Function সমান দৈর্ঘ্য চায়)।
- **`normalize('NFC')`**: বাংলা বা অন্য ভাষার একই দেখতে লেখা ভিন্ন Unicode ক্রমে আসতে পারে (অধ্যায় ১৭); Normalize না করলে সঠিক Password-ও না মিলতে পারে।

- **Session ID ও CSRF Token** দুটিই `randomBytes(32)` —২৫৬ Bit এলোমেলো, অনুমান অসম্ভব। `Math.random()` নিরাপত্তার কাজে কখনো নয়।
- মেয়াদোত্তীর্ণ Session পড়ার সময়ই মুছে যায়।

src/validate.mjs

validate.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// দাম আসে Server-এর Catalog থেকে; Client-এর পাঠানো দাম কখনো নয়।
export const CATALOG = Object.freeze({
  'cox-3d': Object.freeze({ name: "Cox's Bazar, ৩ দিন", price: 12500 }),
  'sylhet-2d': Object.freeze({ name: 'Sylhet, ২ দিন', price: 8900 }),
  'sundarbans-3d': Object.freeze({ name: 'Sundarbans, ৩ দিন', price: 15800 }),
});

const DATE = /^\d{4}-\d{2}-\d{2}$/;
const DAY = 24 * 60 * 60 * 1000;

function checkDate(text, today) {
  if (typeof text !== 'string' || !DATE.test(text)) {
    return 'তারিখ YYYY-MM-DD আকারে দিন';
  }
  const date = new Date(`${text}T00:00:00Z`);
  if (date.toISOString().slice(0, 10) !== text) {
    return 'তারিখটি ক্যালেন্ডারে নেই';
  }
  const days = (date - Date.parse(today.toISOString().slice(0, 10))) / DAY;
  return days >= 1 && days <= 365 ? null : 'আগামীকাল থেকে এক বছরের মধ্যে';
}

// ...
```

validate.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

export function validateTripRequest(body, today = new Date()) {
  if (typeof body !== 'object' || body === null || Array.isArray(body)) {
    return { ok: false, errors: { body: 'একটি JSON Object পাঠান' } };
  }
  const { packageId, travelers, startDate, note = '' } = body;
  const errors = {};
  if (typeof packageId !== 'string' || !Object.hasOwn(CATALOG, packageId)) {
    errors.packageId = 'অজানা Package';
  }
  if (!Number.isInteger(travelers) || travelers < 1 || travelers > 8) {
    errors.travelers = 'যাত্রী ১ থেকে ৮ জন';
  }
```

```
  const dateError = checkDate(startDate, today);
  if (dateError) {
    errors.startDate = dateError;
  }
  if (typeof note !== 'string' || note.length > 200) {
    errors.note = 'নোট সর্বোচ্চ ২০০ অক্ষরের লেখা';
  }
  if (Object.keys(errors).length > 0) {
    return { ok: false, errors };
  }
  const total = CATALOG[packageId].price * travelers;
  return { ok: true, value: { packageId, travelers, startDate, note, total } };
}
```

এখানে তিনটি নিরাপত্তার সিদ্ধান্ত। দাম `CATALOG` থেকে; Body-তে `total` থাকলেও তা পড়াই হয় না। Body থেকে শুধু চারটি নামের Field নেওয়া হয় (Allowlist), তাই `ownerId`, `isAdmin` বা `__proto__` কোনো প্রভাব ফেলে না। এবং `Object.hasOwn(CATALOG, packageId)` : `packageId` যদি `"toString"` হয়, সাধারণ `CATALOG[packageId]` উত্তরাধিকারের একটি Function ফেরত দিত।

src/db.mjs

db.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const TRIP = `SELECT id, owner_id AS ownerId, package_id AS packageId,
  travelers, start_date AS startDate, note, total FROM trips`;

export function createDatabase(file = ':memory:') {
  const db = new DatabaseSync(file);
  db.exec(`
    CREATE TABLE users (
      id INTEGER PRIMARY KEY, email TEXT UNIQUE NOT NULL,
      hash TEXT NOT NULL,
      role TEXT NOT NULL CHECK (role IN ('customer', 'admin')));
    CREATE TABLE trips (
      id TEXT PRIMARY KEY, owner_id INTEGER NOT NULL REFERENCES users (id),
      package_id TEXT NOT NULL, travelers INTEGER NOT NULL,
      start_date TEXT NOT NULL, note TEXT NOT NULL, total INTEGER NOT NULL);
  `);
  // প্রতিটি Query Parameterized: মান আলাদা যায়, SQL-এর লেখার অংশ হয় না।
  const sql = {
    addUser: db.prepare('INSERT INTO users (email, hash, role) VALUES (?, ?, ?)'),
    userByEmail: db.prepare('SELECT * FROM users WHERE email = ?'),
    addTrip: db.prepare('INSERT INTO trips VALUES (?, ?, ?, ?, ?, ?, ?)'),
    tripById: db.prepare(`${TRIP} WHERE id = ?`),
    tripsOf: db.prepare(`${TRIP} WHERE owner_id = ? ORDER BY rowid`),
    allTrips: db.prepare(`${TRIP} ORDER BY rowid`),
```

```
  };
  const plain = (row) => (row ? { ...row } : null);
  return {
    addUser: (email, hash, role) =>
      Number(sql.addUser.run(email, hash, role).lastInsertRowid),
    userByEmail: (email) => plain(sql.userByEmail.get(email)),
    addTrip: (t) => sql.addTrip.run(t.id, t.ownerId, t.packageId,
      t.travelers, t.startDate, t.note, t.total),
    tripById: (id) => plain(sql.tripById.get(id)),
    tripsFor: (user) => (user.role === 'admin'
      ? sql.allTrips.all()
      : sql.tripsOf.all(user.id)).map(plain),
  };
}
```

প্রতিটি Query আগেই `prepare` করা, এবং প্রতিটি মান `?`-এর মাধ্যমে। Template String (`${TRIP}`) শুধু আমাদের নিজের স্থির লেখা জোড়ে, কখনো ব্যবহারকারীর তথ্য নয়। `tripsFor` Authorization-এর নিয়ম মেনে Query বাছে: গ্রাহক শুধু নিজেরগুলো। `plain` SQLite-এর Prototype-হীন Row-কে সাধারণ Object-এ রূপান্তর করে।

src/http.mjs

http.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
export const SECURITY_HEADERS = Object.freeze({
  'content-security-policy': [
    "default-src 'self'", "script-src 'self'", "object-src 'none'",
    "base-uri 'none'", "frame-ancestors 'none'", "form-action 'self'",
  ].join('; '),
  'x-content-type-options': 'nosniff',
  'referrer-policy': 'no-referrer',
  'cross-origin-opener-policy': 'same-origin',
});

export class HttpError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}

// শুধু application/json, সর্বোচ্চ limit Byte। বড় হলে বাকিটা পড়ে ফেলে দেয়
// (Demo-র সরলতার জন্য; Production-এ Connection বন্ধ করাই ভালো)।
export async function readJson(req, limit = 10_000) {
  if (!(req.headers['content-type'] ?? '').startsWith('application/json')) {
    throw new HttpError(415, 'Content-Type application/json হতে হবে');
  }
  const chunks = [];
  let size = 0;
```

```
  for await (const chunk of req) {
    size += chunk.length;
    if (size <= limit) {
      chunks.push(chunk);
    }
  }
  if (size > limit) {
    throw new HttpError(413, 'অনুরোধ খুব বড়');
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8'));
  } catch {
    throw new HttpError(400, 'অবৈধ JSON');
  }
}

// ...
```

http.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

export function send(res, status, body, headers = {}) {
  res.writeHead(status, { ...SECURITY_HEADERS, ...headers });
  res.end(body);
}

export const sendJson = (res, status, data, headers = {}) =>
  send(res, status, JSON.stringify(data), {
    'content-type': 'application/json; charset=utf-8',
    'cache-control': 'no-store',
    ...headers,
  });

export function parseCookies(header = '') {
  const cookies = Object.create(null);
  for (const part of header.split(';')) {
    const index = part.indexOf('=');
    if (index > 0) {
      cookies[part.slice(0, index).trim()] = part.slice(index + 1).trim();
    }
  }
  return cookies;
}
```

`readJson` তিনটি জিনিস আটকায়: ভুল `Content-Type` (415), বড় Body (413; Memory ভরিয়ে Server বসিয়ে দেওয়ার চেষ্টা), ভাঙা JSON (400)। `Content-Type: application/json` বাধ্যতামূলক করাও CSRF-এর বিরুদ্ধে একটি বাড়তি স্তর: একটি সাধারণ HTML Form এই ধরন পাঠাতে পারে না। প্রতিটি Response-এ `SECURITY_HEADERS`,

এবং API-র Response-এ `no-store`, যাতে ব্যক্তিগত তথ্য কোনো Cache-এ না থাকে।

src/app.mjs

app.mjs · JavaScript অংশ ৬ / 8 (Part 1 of 4) চলবে: Node.js

```
import { readFile } from 'node:fs/promises';
import { randomUUID } from 'node:crypto';
import { HttpError, readJson, send, sendJson, parseCookies } from './http.mjs';
import { verifyPassword } from './passwords.mjs';
import { safeEqual } from './sessions.mjs';
import { validateTripRequest } from './validate.mjs';

// শুধু এই তিনটি File; URL থেকে File-এর পথ তৈরি হয় না (Path Traversal নেই)।
const STATIC = {
  '/': ['index.html', 'text/html; charset=utf-8'],
  '/app.mjs': ['app.mjs', 'text/javascript; charset=utf-8'],
  '/style.css': ['style.css', 'text/css; charset=utf-8'],
};
const PUBLIC = new URL('../public/', import.meta.url);

export function createApp(deps) {
  const { db, sessions, limiter, log, dummyHash, secureCookie = false } = deps;
  const secure = secureCookie ? '; Secure' : '';
  const flags = `HttpOnly; SameSite=Strict; Path=/${secure}`;

  function requireUser(req, { csrf = false } = {}) {
    const sid = parseCookies(req.headers.cookie).sid;
    const session = sid ? sessions.get(sid) : null;
    if (!session) {
      throw new HttpError(401, 'Login প্রয়োজন');
    }
    if (csrf) {
      const origin = req.headers.origin;
      if (origin && origin !== `http://${req.headers.host}`) {
        throw new HttpError(403, 'এই Origin থেকে অনুরোধ গ্রহণযোগ্য নয়');
      }
      if (!safeEqual(req.headers['x-csrf-token'] ?? '', session.csrf)) {
        throw new HttpError(403, 'CSRF Token মেলেনি');
      }
    }
    return { sid, session, user: { id: session.userId, role: session.role } };
  }

// ...
```

`requireUser` প্রতিটি সুরক্ষিত Route-এর প্রবেশদ্বার: Cookie থেকে Session, না পেলে 401। অবস্থা বদলানো Route-এ (`csrf: true`) আরও দুটি যাচাই: `Origin` থাকলে তা আমাদের নিজের হতে হবে, এবং Header-এর CSRF Token Session-এরটির সঙ্গে ধ্রুব-সময়ে মিলতে হবে। Static File-এর জন্য একটি স্থির তালিকা: URL-এর কোনো অংশ দিয়ে File-এর পথ তৈরি হয় না, তাই `/../server.mjs`-এর মতো Path Traversal-এর সুযোগ নেই।

app.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

  async function login(req, res) {
    const { email, password } = await readJson(req);
    if (typeof email !== 'string' || typeof password !== 'string') {
      throw new HttpError(400, 'email ও password লেখা হতে হবে');
    }
    const key = email.trim().toLowerCase();
    if (limiter.blocked(key)) {
      throw new HttpError(429, 'অনেকবার ব্যর্থ; কিছুক্ষণ পরে চেষ্টা করুন');
    }
    const user = db.userByEmail(key);
    // ব্যবহারকারী না থাকলেও একই হিসাব, যাতে সময় দেখে Email অনুমান করা না যায়।
    const valid = await verifyPassword(password, user?.hash ?? dummyHash);
    if (!user || !valid) {
      limiter.fail(key);
      log('login.failed', { email: key });
      throw new HttpError(401, 'Email বা Password ভুল');
    }
    limiter.reset(key);
    const { id, csrf } = sessions.create(user);
    log('login.ok', { userId: user.id });
    sendJson(res, 200, { email: user.email, role: user.role, csrf },
      { 'set-cookie': `sid=${id}; ${flags}` });
  }

  async function createTrip(req, res) {
    const { user } = requireUser(req, { csrf: true });
    const result = validateTripRequest(await readJson(req));
    if (!result.ok) {
      sendJson(res, 422, { errors: result.errors });
      return;
    }
    // মালিক আসে Session থেকে; Body-র ownerId বা total গ্রহণ করা হয় না।
    const trip = { id: randomUUID(), ownerId: user.id, ...result.value };
    db.addTrip(trip);
    log('trip.created', { tripId: trip.id, userId: user.id });
    sendJson(res, 201, { trip });
  }

// ...
```

Login-এ চারটি সূক্ষ্মতা। ভুল Password ও অজানা Email—দুটিতেই হুবহু একই বার্তা ও Status, যাতে কোন Email নিবন্ধিত তা জানা না যায়। অজানা Email-এও একটি Dummy Hash-এর সঙ্গে যাচাই চলে, যাতে উত্তরের সময়ও একই রকম থাকে। Limiter একটি Email-এ পাঁচবার ব্যর্থতার পর Password অনুমান থামায়। এবং Log-এ Email ও User ID যায়, Password বা Token কখনো নয়।

app.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  function readTrip(req, res, id) {
    const { user } = requireUser(req);
    const trip = db.tripById(id);
    // অন্যের Trip-এর জন্যও 404, 403 নয়: অস্তিত্বও প্রকাশ করা হয় না।
    if (!trip || (trip.ownerId !== user.id && user.role !== 'admin')) {
      throw new HttpError(404, 'পাওয়া যায়নি');
    }
    sendJson(res, 200, { trip });
  }

  async function route(req, res, pathname) {
    const key = `${req.method} ${pathname}`;
    const tripId = /^\/api\/trips\/([\w-]{1,64})$/.exec(pathname)?.[1];
    if (req.method === 'GET' && Object.hasOwn(STATIC, pathname)) {
      const [file, type] = STATIC[pathname];
      const body = await readFile(new URL(file, PUBLIC));
      send(res, 200, body, { 'content-type': type });
    } else if (key === 'POST /api/login') {
      await login(req, res);
    } else if (key === 'GET /api/session') {
      const { email, role, csrf } = requireUser(req).session;
      sendJson(res, 200, { email, role, csrf });
    } else if (key === 'POST /api/logout') {
      sessions.destroy(requireUser(req, { csrf: true }).sid);
      sendJson(res, 200, { ok: true },
        { 'set-cookie': `sid=; Max-Age=0; ${flags}` });
    } else if (key === 'GET /api/trips') {
      sendJson(res, 200, { trips: db.tripsFor(requireUser(req).user) });
    } else if (key === 'POST /api/trips') {
      await createTrip(req, res);
    } else if (req.method === 'GET' && tripId) {
      readTrip(req, res, tripId);
    } else {
      throw new HttpError(404, 'পাওয়া যায়নি');
    }
  }

// ...
```

app.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  return async function handle(req, res) {
    const { pathname } = new URL(req.url, 'http://localhost');
    try {
      await route(req, res, pathname);
    } catch (error) {
      if (error instanceof HttpError) {
        sendJson(res, error.status, { error: error.message });
        return;
      }
      // বিস্তারিত শুধু Server-এর Log-এ; ব্যবহারকারী পান সাধারণ বার্তা।
      log('server.error', { message: error.message, path: pathname });
      sendJson(res, 500, { error: 'Server-এ একটি সমস্যা হয়েছে' });
    }
  };
}
```

`/api/session` একটি `GET` Route যা CSRF Token ফেরত দেয়, যাতে Page পুনরায় লোড হলে UI আবার Token পায়। এটি নিরাপদ কারণ অন্য Origin-এর Script Same-Origin Policy-র কারণে এই Response পড়তে পারে না, এবং আমরা কোনো CORS Header পাঠাই না। শেষ অংশে অপ্রত্যাশিত Error-এর বিস্তারিত শুধু Log-এ; ব্যবহারকারী পান একটি সাধারণ বার্তা।

src/limiter.mjs, src/log.mjs ও src/start.mjs

limiter.mjs · JavaScript চলবে: Node.js

```
// নির্দিষ্ট সময়-জানালায় একটি Key-এর ব্যর্থ চেষ্টা গোনে (Login-এর জন্য)।
export function createLimiter(options = {}) {
  const { max = 5, windowMs = 15 * 60 * 1000, now = Date.now } = options;
  const failures = new Map();
  const current = (key) => {
    const entry = failures.get(key);
    if (!entry || now() - entry.start >= windowMs) {
      return { count: 0, start: now() };
    }
    return entry;
  };
  return {
    blocked: (key) => current(key).count >= max,
    fail(key) {
      const entry = current(key);
      failures.set(key, { count: entry.count + 1, start: entry.start });
    },
    reset: (key) => failures.delete(key),
```

```
  };
}
```

log.mjs · JavaScript চলবে: Node.js

```
const SENSITIVE = /pass|token|secret|cookie|authorization|csrf/i;

// Log-এ যাওয়ার আগে সংবেদনশীল Key-এর মান মুছে দেয় (Nested Object-সহ)।
export function redact(value) {
  if (Array.isArray(value)) {
    return value.map(redact);
  }
  if (typeof value !== 'object' || value === null) {
    return value;
  }
  return Object.fromEntries(Object.entries(value).map(([key, inner]) =>
    [key, SENSITIVE.test(key) ? '[REDACTED]' : redact(inner)]));
}

export const createLogger = (write = console.log) => (event, details = {}) =>
  write(JSON.stringify({ event, ...redact(details) }));
```

start.mjs · JavaScript চলবে: Node.js

```
import { createServer } from 'node:http';
import { randomBytes } from 'node:crypto';
import { createApp } from './app.mjs';
import { createDatabase } from './db.mjs';
import { createSessions } from './sessions.mjs';
import { createLimiter } from './limiter.mjs';
import { createLogger } from './log.mjs';
import { hashPassword } from './passwords.mjs';

// শুধু 127.0.0.1-এ শোনে: এই কম্পিউটারের বাইরে থেকে পৌঁছানো যায় না।
export async function startServer(options = {}) {
  const { port = 0, users = [], log = createLogger() } = options;
  const db = createDatabase();
  for (const { email, password, role } of users) {
    db.addUser(email, await hashPassword(password), role);
  }
  const dummyHash = await hashPassword(randomBytes(16).toString('hex'));
  const app = createApp({
    db, sessions: createSessions(), limiter: createLimiter(), log, dummyHash,
  });
  const server = createServer(app);
  await new Promise((resolve) => server.listen(port, '127.0.0.1', resolve));
  return { server, url: `http://127.0.0.1:${server.address().port}` };
}
```

server.mjs · JavaScript — চলবে: Node.js

```
import { randomBytes } from 'node:crypto';
import { startServer } from './src/start.mjs';

// Demo Password Code-এ লেখা নেই: Environment Variable, নয়তো প্রতি চালানোয় নতুন।
const password = process.env.SECURETRIP_DEMO_PASSWORD
  ?? randomBytes(9).toString('base64url');
const users = [
  { email: 'rafi@example.com', role: 'customer' },
  { email: 'mina@example.com', role: 'customer' },
  { email: 'admin@example.com', role: 'admin' },
].map((user) => ({ ...user, password }));

const port = Number(process.env.PORT ?? 3020);
const { url } = await startServer({ port, users });
console.log(`SecureTrip (শিক্ষামূলক Demo, শুধু এই কম্পিউটারে): ${url}`);
console.log(`ব্যবহারকারী: ${users.map((user) => user.email).join(', ')}`);
if (!process.env.SECURETRIP_DEMO_PASSWORD) {
  console.log(`এই চালানোর Demo Password: ${password}`);
}
```

`listen(port, '127.0.0.1')` : Server শুধু এই কম্পিউটারের ভেতর থেকে পৌঁছানো যায়, একই Wi-Fi-র অন্য যন্ত্র থেকে নয়। Demo Password Code-এ লেখা নেই; Environment Variable থেকে, নয়তো প্রতিবার নতুন এলোমেলো, এবং শুধু নিজের Terminal-এ দেখানো হয়।

Browser UI: public/index.html ও public/app.mjs

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SecureTrip</title>
  <link rel="stylesheet" href="/style.css">
  <script type="module" src="/app.mjs"></script>
</head>
<body>
  <h1>SecureTrip <small>শিক্ষামূলক Demo</small></h1>
  <form id="login">
    <label>Email <input id="email" type="email" required></label>
    <label>Password <input id="password" type="password" required></label>
    <button id="login-btn">Login</button>
  </form>
  <form id="trip" hidden>
    <label>Package <select id="package">
```

```
      <option value="cox-3d">Cox's Bazar, ৩ দিন</option>
      <option value="sylhet-2d">Sylhet, ২ দিন</option>
      <option value="sundarbans-3d">Sundarbans, ৩ দিন</option>
    </select></label>
    <label>যাত্রী <input id="travelers" type="number" min="1" max="8"
      value="2"></label>
    <label>তারিখ <input id="date" type="date" required></label>
    <label>নোট <input id="note" maxlength="200"></label>
    <button id="add">অনুরোধ পাঠান</button>
  </form>
  <p id="status" role="status"></p>
  <ul id="trips"></ul>
</body>
</html>
```

app.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
const $ = (selector) => document.querySelector(selector);
let csrf = '';

// CSRF Token শুধু এই Variable-এ (Memory); localStorage-এ নয়।
async function api(path, { method = 'GET', body } = {}) {
  const response = await fetch(path, {
    method,
    headers: { 'content-type': 'application/json', 'x-csrf-token': csrf },
    body: body && JSON.stringify(body),
  });
  const data = await response.json();
  if (!response.ok) {
    const details = Object.values(data.errors ?? {}).join(', ');
    throw new Error(data.error ?? details);
  }
  return data;
}

// সব লেখা textContent দিয়ে: নোটে HTML থাকলেও তা লেখা হিসেবেই দেখায়।
function renderTrips(trips) {
  $('#trips').replaceChildren(...trips.map((trip) => {
    const item = document.createElement('li');
    item.textContent = `${trip.packageId} · ${trip.travelers} জন · `
      + `${trip.startDate} · ৳${trip.total} · ${trip.note}`;
    return item;
  }));
}

async function showApp(session) {
  csrf = session.csrf;
  $('#login').hidden = true;
  $('#trip').hidden = false;
```

```
  $('#status').textContent = `Login: ${session.email} (${session.role})`;
  renderTrips((await api('/api/trips')).trips);
}

// ...
```

app.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

$('#login').addEventListener('submit', async (event) => {
  event.preventDefault();
  try {
    const body = { email: $('#email').value, password: $('#password').value };
    $('#password').value = '';
    await showApp(await api('/api/login', { method: 'POST', body }));
  } catch (error) {
    $('#status').textContent = error.message;
  }
});

$('#trip').addEventListener('submit', async (event) => {
  event.preventDefault();
  try {
    const body = {
      packageId: $('#package').value,
      travelers: Number($('#travelers').value),
      startDate: $('#date').value,
      note: $('#note').value,
    };
    const { trip } = await api('/api/trips', { method: 'POST', body });
    $('#status').textContent = `অনুরোধ গৃহীত, মোট ৳${trip.total}`;
    renderTrips((await api('/api/trips')).trips);
  } catch (error) {
    $('#status').textContent = error.message;
  }
});

api('/api/session').then(showApp, () => {});
```

style.css · CSS চলবে: Browser

```
body {
  font-family: system-ui, "Nirmala UI", sans-serif;
  max-width: 40rem;
  margin: 2rem auto;
  padding: 0 1rem;
}
```

```
form {
  display: grid;
  gap: 0.5rem;
  margin-bottom: 1rem;
}

label {
  display: grid;
  gap: 0.25rem;
}

#status {
  color: #1d4ed8;
}
```

HTML-এ কোনো Inline Script বা `onclick` নেই; সব JavaScript একটি বাইরের File-এ, যা `script-src 'self'` CSP-র সঙ্গে খাপ খায়। Trip-এর প্রতিটি লেখা `textContent` দিয়ে, CSRF Token শুধু একটি Variable-এ। UI-র `maxlength` বা `min` / `max` শুধু ব্যবহারকারীর সুবিধা; আসল নিয়ম Server-এর `validateTripRequest`।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd secure-trip
node server.mjs
node --test
```

`node server.mjs` চালালে Terminal-এ ঠিকানা (`http://127.0.0.1:3020`), তিনটি Demo ব্যবহারকারী এবং সেই চালানোর এলোমেলো Password দেখায়; Browser-এ ঠিকানাটি খুলে Login করুন। `node --test` ২৪টি Test চালায়: Unit Test ১১টি (Password, Validation, Session, Limiter, Redaction) এবং আসল HTTP Server-এর বিরুদ্ধে Integration Test ১৩টি—Security Header, একই রকম Login-ব্যর্থতার বার্তা, Cookie-র Flag, CSRF Token ও Origin, Server-এর দাম ও মালিক, Object-level Authorization, 413/415/400, SQL Injection-এর চেষ্টা, Login-এর সীমা, Logout, Static File-এর সীমা, এবং Log-এ কোনো Password বা Token নেই। এছাড়া এই বইয়ের QA-তে আসল Chrome-এ একটি End-to-end যাচাই চলেছে: UI দিয়ে Login, `document.cookie`-তে Session ID দেখা যায় না, `<img onerror>`-যুক্ত নোট শুধু লেখা হিসেবে দেখায় এবং কোনো Handler চলে না, এবং Page-এ ঢোকানো Inline Script CSP আটকে দেয়।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Cannot find module 'node:sqlite'`	Node.js পুরোনো	22.5 বা নতুন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Login-এর পর প্রতিটি অনুরোধে 401	Cookie যায়নি (ভিন্ন Host, যেমন localhost বনাম 127.0.0.1)	একই ঠিকানা ব্যবহার করুন
অনুরোধ পাঠাতে 403	CSRF Token নেই বা পুরোনো	আবার Login, অথবা `/api/session`
429	পাঁচবার ভুল Password	১৫ মিনিট, অথবা Server আবার চালু
Page-এ কিছুই চলে না	Inline Script যোগ করেছেন; CSP আটকাচ্ছে	Script বাইরের File-এ

২০.১৩ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Client-side Validation-কে নিরাপত্তা ভাবা	Browser ব্যবহারকারীর নিয়ন্ত্রণে	Server-এ পুনরায় Validation
ব্যবহারকারীর লেখা `innerHTML`-এ	XSS	`textContent`, Sanitizer
শুধু `<script>` আটকানো	`onerror`, `javascript:` URL	প্রসঙ্গভিত্তিক Encoding, CSP
SQL-এ লেখা জোড়া লাগানো	SQL Injection	Parameterized Query
শুধু Login যাচাই, মালিকানা নয়	IDOR	প্রতিটি Object-এ Authorization
Body থেকে দাম বা `ownerId` নেওয়া	ইচ্ছামতো দাম, অন্যের নামে কাজ	Server-এর উৎস, Session
SHA-256 বা MD5-এ Password	দ্রুত অনুমান	scrypt, Argon2id, bcrypt
Token `localStorage`-এ	XSS পড়ে নিতে পারে	HttpOnly Cookie
CORS-এ যেকোনো Origin ফেরত	অন্য Site Cookie-সহ পড়তে পারে	স্থির Allowlist
Secret Code বা Git-এ	ফাঁস	Environment, Secret Manager
Log-এ Password বা Token	Log-ও ফাঁস হয়	Redaction
ব্যবহারকারীকে Stack Trace দেখানো	ভেতরের তথ্য প্রকাশ	সাধারণ বার্তা, বিস্তারিত Log-এ

২০.১৪ Debugging নির্দেশিকা

১. **Network Panel:** প্রতিটি Request-এর Status, Request ও Response Header দেখুন; Cookie পাঠানো হয়েছে কি না (Request Header-এ `Cookie`), এবং `Set-Cookie`-র Flag।

২. **Application Panel-এর Cookies অংশ:** HttpOnly, Secure, SameSite কলাম।

৩. **Console-এ CSP বার্তা:** "...violates the following Content Security Policy directive" মানে CSP কিছু আটকেছে; Inline Code সরিয়ে বাইরের File-এ নিন, CSP শিথিল করবেন না।
৪. **401 বনাম 403 বনাম 404:** 401 = পরিচয় নেই; 403 = পরিচয় আছে কিন্তু এই কাজের অনুমতি নেই (এখানে CSRF); 404 = নেই, বা আপনার নয়।
৫. **Server Log:** `login.failed`, `server.error`—Redaction-সহ।
৬. **নিজের Code-এ নিরাপত্তার Test:** প্রতিটি ঠিক করা দুর্বলতার জন্য একটি Test, যাতে তা ফিরে না আসে।

২০.১৫ পেশাদার অভ্যাস

- **কখনো Client-কে বিশ্বাস নয়:** Server-এ Validation, দাম, মালিকানা।
- **Allowlist, Blocklist নয়:** Field, Package, Protocol, Sort Column।
- **Defense in Depth:** Encoding এবং CSP; SameSite এবং CSRF Token; একটি স্তর ব্যর্থ হলে আরেকটি।
- **Least Privilege:** প্রতিটি ব্যবহারকারী, Database Account ও Token-এর ন্যূনতম অধিকার।
- **নিজে Crypto না বানানো:** Node.js-এর `crypto`, পরীক্ষিত Library।
- **ব্যর্থতায় বন্ধ (Fail Closed):** সন্দেহ হলে প্রত্যাখ্যান।
- **নিরাপত্তার Test** স্বাভাবিক Test-এর মতোই; প্রতিটি নিয়ন্ত্রণ প্রমাণিত।
- **OWASP Top 10 ও ASVS** নিয়মিত পড়া; Dependency হালনাগাদ।

২০.১৬ অনুশীলনী

প্রাথমিক স্তর

১. `escapeHtml` দিয়ে `Tom & "Jerry" <b>`-কে HTML-এ নিরাপদ করুন, এবং ফল দেখান।
২. একজন সহকর্মী বলছেন, "Form-এ `required` ও `maxlength` দিয়েছি, Server-এ যাচাই লাগবে না।" দুই বাক্যে উত্তর দিন।
৩. প্রতিটি Authentication না Authorization: (ক) Password যাচাই, (খ) Trip-এর মালিক যাচাই, (গ) Session-এর মেয়াদ, (ঘ) Admin-ই সব দেখতে পারেন।
৪. একটি Session Cookie-র `Set-Cookie` Header লিখুন, Production-এর জন্য (HTTPS)।
৫. নিচের Code-এর নিরাপত্তা-ত্রুটি খুঁজুন: `fetch('https://api.maps.example/geo?key=sk_live_93ab...')`, একটি Browser Bundle-এ।

মধ্যম স্তর

৬. `safeHref(url)` লিখুন, যা শুধু `https:`, `http:` ও Relative Link গ্রহণ করে, বাকিগুলোর জন্য `'#'` দেয়।

৭. `node:sqlite`-এ Place-এর নামে আংশিক খোঁজ (`LIKE`) Parameter দিয়ে করুন, যাতে ব্যবহারকারীর `%` চিহ্ন Wildcard হিসেবে কাজ না করে।

৮. একটি `corsHeaders(origin)` লিখুন: স্থির Allowlist-এর Origin হলে Header দেয়, নয়তো কিছু নয়।

৯. SecureTrip-এর `redact`-কে বাড়ান, যাতে `email` পুরো না মুছে আংশিক ঢাকে (`ra***@example.com`)।

চ্যালেঞ্জ স্তর

১০. একটি `safeMerge` লিখুন, যা Prototype-হীন Object-এ মেশায়, বিপজ্জনক Key বাদ দেয়, এবং ৫ স্তরের বেশি গভীরতায় Error দেয়।

১১. SecureTrip-এর Limiter দিয়ে Email ও IP দুটির জন্যই সীমা প্রয়োগের একটি Function লিখুন, এবং দেখান যে একটি IP বিভিন্ন Email-এ চেষ্টা করলেও আটকে যায়।

১২. নিচের Express-ধাঁচের Route-এর অন্তত পাঁচটি নিরাপত্তা-ত্রুটি চিহ্নিত করুন এবং প্রতিটির সমাধান লিখুন:
`app.get('/trip/delete', (req, res) => { db.exec("DELETE FROM trips WHERE id = " + req.query.id); res.send('<p>' + req.query.id + ' মুছে ফেলা হয়েছে</p>'); });`

২০.১৭ অনুশীলনীর সমাধান

সমাধান ১

escape.js · JavaScript — চলবে: Browser Console ও Node.js

```
const HTML_ESCAPES = {
  '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;',
};
const escapeHtml = (text) =>
  String(text).replace(/[&<>"']/g, (ch) => HTML_ESCAPES[ch]);

console.log(escapeHtml('Tom & "Jerry" <b>'));
```

Output

```
Tom &amp; &quot;Jerry&quot; &lt;b&gt;
```

`&` সবার আগে নয়, একবারে সব চিহ্ন বদলানো হয়েছে; ফলে আগে বদলানো `<`-এর `&` আবার বদলায় না।

সমাধান ২ — Browser ব্যবহারকারীর নিয়ন্ত্রণে: DevTools-এ `maxlength` মুছে ফেলা যায়, অথবা Browser ছাড়াই `curl` দিয়ে যেকোনো Request পাঠানো যায়। তাই Client-এর যাচাই ব্যবহারকারীর সুবিধা, আর Server-এর যাচাই নিরাপত্তা—দ্বিতীয়টি সব সময় লাগবে।

সমাধান ৩ — (ক) Authentication, (খ) Authorization (Object-level), (গ) Authentication (পরিচয় এখনো বৈধ কি না), (ঘ) Authorization (Role-ভিত্তিক)।

সমাধান ৪

Terminal

```
Set-Cookie: sid=<৩২ Byte এলোমেলো, base64url>; HttpOnly; Secure;
  SameSite=Lax; Path=/; Max-Age=1800
```

`HttpOnly` JavaScript থেকে লুকায়, `Secure` শুধু HTTPS-এ পাঠায়, `SameSite` অন্য Site-এর Request-এ বাধা, `Max-Age` ৩০ মিনিটে শেষ। অন্য Site থেকে আসা Link-এ ক্লিক করেও ব্যবহারকারী Login অবস্থায় থাকতে চাইলে `Lax`; SecureTrip-এর মতো কড়া হলে `Strict`।

সমাধান ৫ — Browser Bundle-এ থাকা যেকোনো Key Public: যে কেউ DevTools-এর Network বা Source থেকে তা পড়ে নিজের কাজে ব্যবহার করতে পারে (এবং বিল আপনার নামে)। সমাধান: Secret Key-যুক্ত Request নিজের Server থেকে (Server Proxy), অথবা Provider যদি Browser-এর জন্য বিশেষ সীমিত Key দেয় (নির্দিষ্ট Domain-এ সীমাবদ্ধ), সেটি। Key ইতিমধ্যে প্রকাশিত হলে তা বাতিল করে নতুন Key (Rotate)।

সমাধান ৬

safe-href.js · JavaScript — চলবে: Browser Console ও Node.js

```
function safeHref(url) {
  try {
    const parsed = new URL(url, 'https://trip.example');
    return ['https:', 'http:'].includes(parsed.protocol) ? url : '#';
  } catch {
    return '#';
  }
}

const inputs = ['https://ok.example/a', '/trips/7', 'JavaScript:alert(1)',
  ' javascript:void(0)', 'data:text/html,hi', 'http://[bad'];
console.log(inputs.map(safeHref).join(' | '));
```

Output

```
https://ok.example/a | /trips/7 | # | # | # | #
```

বড় হাতের `JavaScript:` এবং সামনে ফাঁকা স্থান—দুটিই Browser `javascript:` হিসেবে মানে; `URL` Parser একই নিয়মে Protocol বের করে, তাই নিজের Regex-এর চেয়ে নির্ভরযোগ্য।

সমাধান ৭

like.mjs · JavaScript — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE places (name TEXT);
  INSERT INTO places VALUES ('Sylhet'), ('Cox''s Bazar'), ('100% Sundarbans');`);

const escapeLike = (text) => text.replace(/[\\%_]/g, (ch) => `\\${ch}`);
const search = db.prepare(
  "SELECT name FROM places WHERE name LIKE ? ESCAPE '\\'");

for (const input of ['ba', '%', "'"]) {
  const rows = search.all(`%${escapeLike(input)}%`);
  console.log(JSON.stringify(input), rows.map((row) => row.name).join(', '));
}
```

Output

```
"ba" Cox's Bazar, 100% Sundarbans
"%" 100% Sundarbans
"'" Cox's Bazar
```

Parameter SQL Injection আটকায় (`'` নিরাপদ), কিন্তু `LIKE`-এর ভেতরে `%` ও `_` নিজস্ব অর্থ রাখে। Escape না করলে `%` খুঁজলে সব সারি আসত। (SQLite-এর `LIKE` ASCII-তে ছোট-বড় হাতের পার্থক্য করে না, তাই `ba` দিয়ে `Bazar` ও `Sundarbans` দুটিই মিলেছে।)

সমাধান ৮

cors.js · JavaScript — চলবে: Node.js

```
const ALLOWED = new Set([
  'https://app.trip.example',
  'https://admin.trip.example',
]);

function corsHeaders(origin) {
  if (!ALLOWED.has(origin)) {
    return {};
  }
  return {
    'access-control-allow-origin': origin,
    'access-control-allow-credentials': 'true',
    vary: 'Origin',
  };
```

```
}

console.log(corsHeaders('https://app.trip.example'));
console.log(corsHeaders('https://app.trip.example.evil.test'));
console.log(corsHeaders(undefined));
```

Output

```
{
  'access-control-allow-origin': 'https://app.trip.example',
  'access-control-allow-credentials': 'true',
  vary: 'Origin'
}
{}
{}
```

হুবহু মিল, `startsWith` বা `includes` নয়: দ্বিতীয় উদাহরণটি `startsWith` পরীক্ষা পেরিয়ে যেত। `Vary: Origin` Cache-কে বলে যে Response Origin-ভেদে আলাদা।

সমাধান ৯

mask-email.mjs · JavaScript চলবে: Node.js

```
import { redact } from '../../project/secure-trip/src/log.mjs';

const maskEmail = (email) => email.replace(/^(.{1,2})[^@]*(@.*)$/u, '$1***$2');

function redactForLog(details) {
  const clean = redact(details);
  return typeof clean.email === 'string'
    ? { ...clean, email: maskEmail(clean.email) }
    : clean;
}

console.log(redactForLog({ email: 'rafi@example.com', password: 'x' }));
console.log(redactForLog({ email: 'a@b.example' }));
```

Output

```
{ email: 'ra***@example.com', password: '[REDACTED]' }
{ email: 'a***@b.example' }
```

সমাধান ১০

safe-merge.mjs · JavaScript চলবে: Node.js

```
const BLOCKED = new Set(['__proto__', 'constructor', 'prototype']);

function safeMerge(target, source, depth = 0) {
  if (depth > 5) {
    throw new RangeError('অনেক গভীর Object');
  }
  for (const [key, value] of Object.entries(source)) {
    if (BLOCKED.has(key)) {
      continue;
    }
    const isObject = typeof value === 'object' && value !== null;
    const existing = Object.hasOwn(target, key)
      ? target[key]
      : Object.create(null);
    target[key] = isObject ? safeMerge(existing, value, depth + 1) : value;
  }
  return target;
}

const text = '{"a":{"b":1},"__proto__":{"x":1},"constructor":{"y":2}}';
const body = JSON.parse(text);
const merged = safeMerge(Object.create(null), body);
console.log(JSON.stringify(merged), ({}).x, Object.getPrototypeOf(merged));

let deep = {};
for (let i = 0; i < 8; i += 1) {
  deep = { inner: deep };
}
try {
  safeMerge(Object.create(null), deep);
} catch (error) {
  console.log(error.name, error.message);
}
```

Output

```
{"a":{"b":1}} undefined null
RangeError অনেক গভীর Object
```

সমাধান ১১

dual-limit.mjs · JavaScript চলবে: Node.js

```
import { createLimiter } from '../../project/secure-trip/src/limiter.mjs';

const byEmail = createLimiter({ max: 5 });
const byIp = createLimiter({ max: 20 });

function attempt(ip, email, success) {
  if (byIp.blocked(ip) || byEmail.blocked(email)) {
    return 429;
  }
  if (success) {
    byEmail.reset(email);
    return 200;
  }
  byIp.fail(ip);
  byEmail.fail(email);
  return 401;
}

const statuses = [];
for (let i = 0; i < 22; i += 1) {
  statuses.push(attempt('10.0.0.7', `user${i}@example.com`, false));
}
console.log(statuses.slice(-3).join(' '));
console.log(attempt('10.0.0.8', 'user1@example.com', true));
```

Output

```
401 429 429
200
```

একটি IP প্রতিবার ভিন্ন Email দিয়ে চেষ্টা করলে (Password Spraying) Email-ভিত্তিক সীমা কাজে আসে না, কিন্তু IP-ভিত্তিক সীমা বিশটির পর আটকায়। অন্য IP থেকে সঠিক Login তখনো চলে। বাস্তবে IP ভাগ করা (একই অফিস) হতে পারে, তাই IP-র সীমা ঢিলে রাখা হয়।

সমাধান ১২ — (১) অবস্থা বদলানো কাজ `GET`-এ: একটি `<img src="/trip/delete?id=5">` যুক্ত যেকোনো Page তা চালাতে পারে (CSRF); সমাধান `DELETE` বা `POST`, CSRF Token। (২) SQL-এ লেখা জোড়া লাগানো: `id=1 OR 1=1` সব মুছে দেবে; সমাধান Parameterized Query। (৩) Authentication নেই। (৪) Authorization নেই: যে কেউ যেকোনো Trip মুছতে পারে; সমাধান মালিক বা Admin যাচাই। (৫) Response-এ `req.query.id` সরাসরি HTML-এ: Reflected XSS; সমাধান Escape বা JSON Response। (৬) `id`-এর Validation নেই; সমাধান ধরন ও আকারের Allowlist। (৭) কোনো Log নেই, যাতে পরে বোঝা যায় কে কী মুছেছে।

২০.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Client-side Validation-এর মূল উদ্দেশ্য?

ক) নিরাপত্তা

খ) ব্যবহারকারীকে তাৎক্ষণিক ভুল দেখানো

গ) Server-এর কাজ বাদ দেওয়া

ঘ) SQL Injection প্রতিরোধ

প্রশ্ন ২. `innerHTML`-এ `<img src="data:," onerror="...">` বসালে কী ঘটে?

ক) কিছুই না, কারণ `<script>` নেই

খ) শুধু লেখা দেখায়

গ) `onerror` Handler চলে

ঘ) Browser Error দিয়ে থেমে যায়

প্রশ্ন ৩. SQL Injection-এর সঠিক প্রতিরোধ?

ক) Parameterized Query

খ) Input থেকে `'` মুছে ফেলা

গ) Input বড় হাতের অক্ষরে রূপান্তর

ঘ) Client-side Validation

প্রশ্ন ৪. Login করা গ্রাহক URL-এর ID বদলে অন্যের Trip দেখতে পাচ্ছেন। এটি কোন ধরনের ত্রুটি?

ক) XSS

খ) CSRF

গ) SQL Injection

ঘ) Broken Access Control (IDOR)

প্রশ্ন ৫. Password সংরক্ষণের জন্য উপযুক্ত?

ক) SHA-256

খ) Base64

গ) Salt-সহ scrypt বা Argon2id

ঘ) AES Encryption, Key Code-এ

প্রশ্ন ৬. `HttpOnly` Cookie Flag কী করে?

ক) JavaScript-কে Cookie পড়তে দেয় না

খ) শুধু HTTP-তে পাঠায়, HTTPS-এ নয়

গ) CSRF পুরোপুরি বন্ধ করে

ঘ) Cookie-কে Encrypt করে

প্রশ্ন ৭. CORS সম্পর্কে কোনটি সঠিক?

ক) এটি API-কে সব অননুমোদিত অনুরোধ থেকে রক্ষা করে

খ) এটি Browser-কে জানায় কোন Origin Response পড়তে পারবে

গ) `*` দিলে Cookie-সহ API নিরাপদ

ঘ) `curl`-ও CORS মানে

প্রশ্ন ৮. `script-src 'self'` CSP কী আটকায়?

ক) একই Origin-এর Script File

খ) সব Network Request

গ) CSS File

ঘ) Inline Script ও `onerror`-এর মতো Inline Handler

প্রশ্ন ৯. নিচের কোনটি Prototype Pollution ঘটাতে পারে?

ক) `JSON.parse` একা

খ) `__proto__` Key বাদ না দিয়ে Recursive Merge

গ) `Object.freeze`

ঘ) `Map`-এ `set('__proto__', 1)`

প্রশ্ন ১০. অন্যের Trip-এর অনুরোধে SecureTrip 403-এর বদলে 404 দেয় কেন?

ক) 404 দ্রুত

খ) 403 HTTP-তে নেই

গ) Trip-এর অস্তিত্বও প্রকাশ না করতে

ঘ) Browser 403 দেখাতে পারে না

২০.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–গ · ৬–ক · ৭–খ · ৮–ঘ · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	নিরাপত্তা আসে Server-এর যাচাই থেকে।
২	গ	অনুচ্ছেদ ২০.৩-এ Chrome-এ প্রমাণিত; `<script>` না চললেও Handler চলে।
৩	ক	মান কখনো SQL-এর লেখা হয় না; চিহ্ন মোছা অসম্পূর্ণ ও ভঙ্গুর।
৪	ঘ	Authentication ঠিক, Object-level Authorization নেই।
৫	গ	ধীর, Salt-যুক্ত, Memory-নির্ভর; দ্রুত Hash অনুমান সহজ করে।
৬	ক	XSS হলেও Session ID পড়া যায় না; CSRF-এর জন্য SameSite ও Token।
৭	খ	CORS Browser-এর পড়ার অনুমতি; সুরক্ষা আসে AuthN ও AuthZ থেকে।
৮	ঘ	একই Origin-এর File চলে; অনুচ্ছেদ ২০.৯-এ Chrome-এ যাচাই।
৯	খ	`target['__proto__']` আসলে `Object.prototype`।
১০	গ	অস্তিত্বহীন ও অন্যের Trip—দুটিতে একই উত্তর।

২০.২০ অধ্যায়ের সারসংক্ষেপ

- Threat Model: কী রক্ষা, কার কাছ থেকে; সবচেয়ে গুরুত্বপূর্ণ Trust Boundary হলো Browser থেকে Server।
- Client-side Validation সুবিধা, নিরাপত্তা নয়; Server-এ Allowlist দিয়ে পুনরায় যাচাই।
- XSS: অবিশ্বস্ত লেখা কখনো Markup নয়; `textContent`, প্রসঙ্গভিত্তিক Encoding, CSP।
- SQL Injection: Parameterized Query; Column-এর নাম Allowlist থেকে।
- Authentication পরিচয়, Authorization প্রতিটি Object-এর অনুমতি; মালিক Session থেকে।
- Password: scrypt বা Argon2id, Salt, `timingSafeEqual`; একই রকম ব্যর্থতার বার্তা; ব্যর্থ চেষ্টার সীমা।
- Session Cookie: HttpOnly, Secure, SameSite; CSRF Token ও Origin যাচাই।
- CORS পড়ার অনুমতি, সুরক্ষা নয়; CSP দ্বিতীয় স্তরের প্রতিরক্ষা।
- Prototype Pollution: বিপজ্জনক Key বাদ, `Map`, `Object.create(null)`, Allowlist।
- Secret Environment-এ, Dependency নিয়মিত যাচাই, Log-এ Redaction।
- শিক্ষামূলক Demo-কে Production-প্রস্তুত বলা যায় না; তার জন্য স্বাধীন Review ও আরও অনেক নিয়ন্ত্রণ লাগে।

পরের অধ্যায়ে Design Pattern ও Software Architecture: বড় Application-কে কীভাবে পরিষ্কার, পরীক্ষাযোগ্য অংশে সাজানো যায়।

২০.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Threat Model	কী রক্ষা, কার কাছ থেকে, কীভাবে—তার বিশ্লেষণ।
Trust Boundary	যেখানে কম বিশ্বস্ত তথ্য বেশি বিশ্বস্ত এলাকায় ঢোকে।
XSS	ব্যবহারকারীর লেখা অন্যের Browser-এ Script হিসেবে চলা।
Output Encoding	প্রসঙ্গ অনুযায়ী লেখাকে শুধু তথ্য হিসেবে নিরাপদ করা।
SQL Injection	ইনপুট SQL-এর অংশ হয়ে যাওয়া।
Authentication	পরিচয় যাচাই।
Authorization	নির্দিষ্ট কাজের অনুমতি যাচাই।
IDOR	ID বদলে অন্যের Object-এ পৌঁছানো।
Salt	প্রতিটি Password-এর আলাদা এলোমেলো মান।
CSRF	অন্য Site থেকে ব্যবহারকারীর Cookie-সহ অনুরোধ করানো।
CORS	অন্য Origin-কে Response পড়ার অনুমতির নিয়ম।
CSP	কোন উৎসের Script চলবে তার Browser-নীতি।
Prototype Pollution	`Object.prototype`-এ অনাকাঙ্ক্ষিত Property লেখা।
Redaction	Log থেকে সংবেদনশীল তথ্য মুছে ফেলা।

২০.২২ আরও পড়ুন

- **OWASP Top 10:** `https://owasp.org/www-project-top-ten/` — Web Application-এর প্রধান ঝুঁকি।
- **OWASP Cheat Sheet Series:** `https://cheatsheetseries.owasp.org/` — XSS, Session, Password Storage, CSRF-এর ব্যবহারিক নির্দেশিকা।
- **MDN — Content Security Policy:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP` — CSP-র Directive।
- **MDN — Set-Cookie:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie` — Cookie-র Flag।
- **Node.js — crypto.scrypt:** `https://nodejs.org/api/crypto.html#cryptoscryptpassword-salt-keylen-options-callback` — Password Hashing।